Presentation Guide — Intelligent Payment Fraud Investigation & Dispute Resolution Agent

Use this as your speaking script + live demo checklist.

---

1. Opening — what problem does it solve? (30 sec)

> "When a customer disputes a card transaction, an analyst has to pull data from many systems — transaction records, device data, location history, merchant risk, dispute history — then check bank policy, and decide how to route the case. This agent automates the *evidence gathering and analysis* part, but — and this is the key design principle — **it never acts autonomously. A human always makes the final disposition.**"

Key phrase to say: "**recommend, don't act — human-in-the-loop.**"

---

2. Architecture in one slide (1 min)

`Intake → Plan → Act → Retrieve → Assess → Recommend → Review → Audit`

Stage	What happens	Code
Intake	Validates mandatory fields, rejects unmasked card numbers	`FraudCaseService.createCase()`
Plan	Classifies the dispute goal, selects which tools to call	`InvestigationPlanner`
Act	Calls only **allow-listed, read-only** tools — each timed, traced, safely-failed	`EvidenceTools` (8 tools)
Retrieve	Searches policy documents, keeps citations with doc/version/section/chunk/score	`LexicalPolicyRetriever`
Assess + Recommend	Produces a typed recommendation (classification, risk, route, rationale, evidence refs)	`FraudCaseProcessor.analyze()`
Review	Human picks APPROVED / REVISED / REROUTED / REJECTED + reason	Review form
Audit	Every state change, tool call, citation, review → append-only hashed audit log	`AuditEventWriter`

---

3. The headline feature — three interchangeable "brains"

(1–2 min)

> "The most interesting design decision: the analysis brain is behind one contract — `FraudCaseProcessor` — and we ship **three implementations**, switched by a single config property."

Mode	How it works	When to use
`rules`	`RulesFraudCaseAgent` — deterministic rules over tool evidence. No model, no API key.	Default, CI tests, offline
`spring-ai`	`SpringAiFraudCaseAgent` — `ChatClient` + typed `.entity()` structured output	Anthropic via Spring AI
`langchain4j`	`LangChain4jFraudCaseAgent` — `AiServices` interface + `RetrievalAugmentor`	Anthropic via LangChain4j

Config: `capstone.processor-mode` in `application.yml` (or `PROCESSOR_MODE` env var).

> "Same case in, same schema out — whichever brain runs it. If the model fails or returns garbage, `RecommendationValidator` rejects it and the case fails **safe** to INCOMPLETE → manual review. It can never make a bad autonomous decision."

---

4. Where Spring AI and LangChain4j are actually used

Show this table — it answers "why both?":

Capability	Spring AI	LangChain4j
Model call	`ChatClient` + `.entity(FraudCaseRecommendation.class)`	`AiServices` interface with `@SystemMessage`
Tool calling	`@Tool` annotations on `EvidenceTools`	*same methods*, second `@Tool` annotation
Memory (per case)	`MessageChatMemoryAdvisor` keyed by `caseId`	`ChatMemoryProvider` keyed by `@MemoryId`
RAG / policy	policy chunks injected into prompt	`PolicyContentRetriever` → `RetrievalAugmentor`

> "One `EvidenceTools` class serves both frameworks — the same 8 methods carry both `@Tool` annotations. Memory is **keyed by case ID**, so two cases can never leak context into each other."

---

5. Safety & auditability (the "enterprise" slide)

- **Tools are read-only + allow-listed** — the agent can't invent new tools or mutate anything.
- **Model never transitions case state** — `FraudCaseService.transition()` owns the state machine; model output is just a recommendation.
- **`humanReviewRequired=true` is enforced** by `RecommendationValidator` — a

recommendation without it is rejected.
- **Every tool call is traced** (hash of args, duration, status — no raw sensitive payloads).
- **Audit log is append-only** — case created, investigated, reviewed, routed: all hashed events.
- **Intake rejects unmasked PANs** and script/markup in statements.
- **Memory is case-scoped** — no cross-case contamination.

---

6. Tech stack

Java 21 · Spring Boot 3.5.14 · Spring MVC + Thymeleaf · Spring Data JPA + H2 (MySQL-ready) · Spring AI 1.1.8 · LangChain4j 1.22.0 · JUnit 5 + Mockito (33 tests)

---

7. LIVE DEMO SCRIPT — follow these exact steps

App should be running at **http://localhost:8090** (`mvn spring-boot:run`, rules mode).

Demo flow

1. **Case list** (`/`) — show dashboard, stat cards, "synthetic data only" framing.
2. **New case** (`/cases/new`) — point out pre-filled fraud scenario. Only explain one field:
    - *"The transaction ref drives the deterministic synthetic backend: `TXN-FRAUD-*`, `TXN-DUP-*`, `TXN-GENUINE-*`, `TXN-MISSING-*`."*
    - Click **Create case**.
3. **Case page** — click ▶ **Run investigation**. Watch status go `NEW` → `READY_FOR_REVIEW` in seconds.
4. **Walk the page top-to-bottom:**
    - *Intake card* — everything masked, no raw account numbers.
    - *Evidence table* — HIGH/MEDIUM severity findings: new device, location anomaly, unrecognized merchant.
    - *Tool trace timeline — "every tool call is traced with timing and a hash — full observability."*
    - *Recommendation card* — `UNAUTHORISED_TRANSACTION` → `FRAUD_INVESTIGATION`, confidence bar, rule R-004 with the exact facts that fired it, policy citations (POL-CNP-017 with section/chunk/score).
5. **Chat box** — ask: *"Which evidence supports the fraud classification?"* — the answer references real evidence IDs, case-scoped only.
6. **Human review** — *"This is the human-in-the-loop point. The agent recommended — now a person disposes."* Pick **APPROVED**, reason "evidence supports unauthorized CNP fraud" → case routes to `ROUTED`.

If asked "can it go wrong?"

> "Yes — that's tested. Bad API key or model failure → the case goes INCOMPLETE and stays in the human queue. The agent degrades safely instead of guessing."

8. API cheat sheet (if asked about integration)

```
POST   /api/cases                                  # create case (intake)
POST   /api/cases/{id}/investigations              # run the 8-stage pipeli
GET    /api/cases/{id}/recommendation              # typed recommendation +
GET    /api/cases/{id}/evidence                    # evidence items w/ seve
GET    /api/cases/{id}/tool-traces                 # tool execution trace
GET    /api/cases/{id}/citations                   # policy citations w/ sc
POST   /api/cases/{id}/messages                    # case-scoped follow-up
POST   /api/cases/{id}/reviews                     # human disposition
GET    /api/cases/{id}/audit-events                # append-only audit trai
```

---

9. Likely questions & one-line answers

- "**Why three modes?**" — Rules = deterministic baseline & tests; the two AI adapters show the same contract works across frameworks — swap with one property.
- "**Is the data real?**" — No, `SyntheticEvidenceAdapter` generates deterministic scenario data — this is a training build.
- "**Can the agent block a card / refund?**" — No. Tools are read-only by design; it only *recommends a route*.
- "**How do you prevent hallucinated evidence?**" — Recommendations must cite evidence IDs and policy chunks that actually exist in context; validation rejects unknown refs.
- "**What does RAG do here?**" — 4 policy docs chunked + indexed (`POL-CNP`, `POL-DUP`, `POL-MRC`, `POL-SLA`); retriever returns top-k with scores, abstains below threshold so the agent can't cite irrelevant policy.
- "**What happens on model failure?**" — Safe failure → status `INCOMPLETE`, goes to manual review. Verified live.
- "**Where's the state machine?**" — `FraudCaseService.transition()` — allowed transitions table; e.g. `ROUTED → CLOSED` only.

---

10. Closing line

> "So the pattern is: **AI does the synthesis, policy does the grounding, tools do the evidence, and the human does the deciding** — with an audit trail over everything. That's what makes it safe enough for a real dispute workflow."